Evaluating Language Models and RAG Systems

Measuring retrieval quality, correctness, and grounding

Purpose

This short reference note is written for learning, semantic retrieval, and grounded question-answering experiments. It uses original explanatory content and is organized into coherent sections rather than unrelated filler.

1. Why Evaluation Is Necessary

A demo can work on a few convenient questions and still fail on realistic use. Evaluation makes quality measurable and helps compare system versions. A useful test set represents expected questions, different phrasings, difficult cases, and questions that the corpus cannot answer. The set should be stable enough that changes in results can be attributed to system changes rather than to changing test questions.

2. Retrieval Metrics

Recall@k asks whether a relevant item appears among the top k retrieved results. Precision@k measures how many of the top k results are relevant. Mean reciprocal rank rewards placing the first relevant result near the top. These metrics require relevance labels or another reliable judgment process. For small projects, manually labeling a modest set of questions can provide useful diagnostic information.

3. Answer Quality

Answer correctness evaluates whether the response answers the question accurately. Completeness checks whether important parts are missing. Groundedness asks whether claims are supported by the retrieved context. Citation accuracy checks whether cited sources actually support the associated claims. These dimensions should not be collapsed into one vague score because a correct answer can be poorly cited, and a well-cited answer can still be incomplete.

4. Supported and Unsupported Questions

A balanced evaluation includes questions with clear answers in the documents and questions whose answers are absent. Unsupported questions test whether the system can abstain rather than invent an answer. Include paraphrases, terminology variations, ambiguous queries, and questions that require combining multiple passages. The test set should not simply repeat the wording of the source headings.

5. Comparing Chunking Strategies

When comparing chunk sizes, keep the evaluation questions and model configuration stable. Record chunk size, overlap, embedding model, top-k, and generation settings. Compare retrieval relevance, groundedness, latency, and any cost differences. A strategy that performs well on one narrow dataset may not generalize, so report the number and type of examples used.

6. Error Analysis and Reproducibility

For every failure, save the question, retrieved chunk IDs, source text, similarity scores if available, prompt, response, and expected behavior. Categorize the error as extraction, chunking, embedding, retrieval, context assembly, or generation. Version the corpus and configuration. A reproducible baseline makes it easier to tell whether a change genuinely improves the system or merely shifts failures elsewhere.

Key Takeaways

Evaluate retrieval and generation separately, include unsupported questions, and keep a reproducible test set. Metrics are most useful when paired with examples that explain why a system failed.

End of document • Use source passages as evidence and verify claims during evaluation.